Browser History Management System

CSS2281 Problem-Based Learning (PBL) – Code Explanation

This document outlines the technical implementation of the Browser History Management system. The application is built in C and utilizes the Last-In, First-Out (LIFO) principle of the Stack data structure to simulate real-world web browser navigation.

1. The Data Structure

The program utilizes a custom struct to handle multiple stacks. This struct relies on a 2D character array rather than a standard integer array, enabling it to store strings (URLs).

- `urls` **array:** A 2D character array capable of holding up to 100 strings (`MAX_HISTORY`), where each string can be up to 256 characters long (`URL_LENGTH`).
- `top` **variable:** An integer that continuously tracks the exact array index of the most recently added URL.

2. Core Stack Operations

The system relies on three fundamental stack operations to manage memory constraints and data insertion/removal:

Function	Description and Logic
`initStack()`	Initializes the stack by setting the `top` variable to `-1` . Because C arrays are 0-indexed, a `top` of -1 mathematically signifies that the stack is entirely empty.
`push()`	First checks for overflow (`isFull()`). If space permits, it increments the `top` index by 1, then executes `strcpy()` to copy the provided URL string into the array at the new top position.
`pop()`	First checks for underflow (`isEmpty()`). If a URL exists, it copies the string currently at the top position into a temporary variable, and then decrements `top` by 1, virtually removing it from the stack.

3. Navigation Logic

Real-world browser navigation requires the coordination of two distinct stacks: a **Back Stack** and a **Forward Stack**. The program orchestrates these stacks based on user actions:

Visiting a New Page: The current active URL is pushed onto the *Back Stack* to save it. The system then loads the new URL as the active page. Crucially, it clears the *Forward Stack* (via `initStack()`) because visiting a brand new page breaks the previous forward navigation chain.

Going Back: The current active URL is pushed onto the *Forward Stack* to preserve it for future forward navigation. The system then pops the most recent URL from the *Back Stack* and sets it as the new active page.

Going Forward: This operation reverses the back action. The current active URL is pushed to the *Back Stack*, and the top URL saved in the *Forward Stack* is popped out to become the active page.